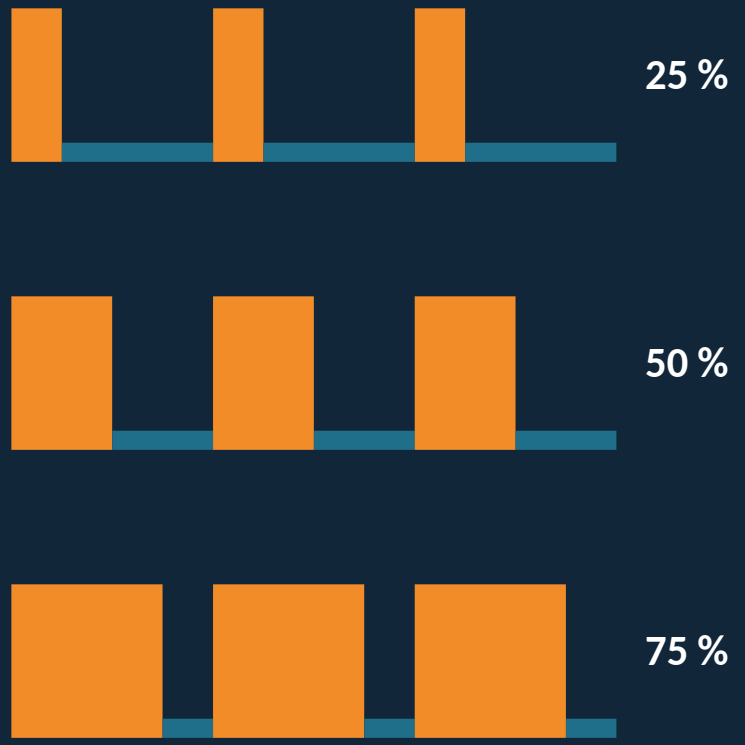
PWM en Raspberry Pi Pico y Pico 2 W

Control de velocidad y sentido de un motor DC con puente H

RP2040 vs RP2350 · registros y punteros · Pico SDK · comando REPL



Ana Cisneros · Leslie Sanchez

Equipo 2 · Periférico: PWM · Actividad integradora

PARTE 1 · INTRODUCCIÓN

Ruta de la presentación

1

Introducción y problema

5 min

2

Antecedentes, aplicaciones y arquitectura

10 min

3

Registros y punteros

7 min

4

Pico SDK

5 min

5

RP2040 vs RP2350

5 min

6

Comando REPL

5 min

7

Demostración física

5–8 min

Total ≈ 42–48 min

con demostración en vivo en las dos tarjetas

necesidad → evolución → periférico → arquitectura → registros → punteros → C → SDK → REPL → motor

PARTE 1 · INTRODUCCIÓN

El problema: mover potencia con un pin

Un pin no alcanza

Salida de 3.3 V y hasta 12 mA por pin (configurable). Un motor DC pide cientos de mA o más y 6–12 V.

Resistencia = calor

Bajar el voltaje con un reóstato disipa la diferencia como calor: I^2R. Poca eficiencia y no es controlable por software.

Queremos velocidad variable

Una señal digital que entregue potencia promedio ajustable, con un puente H que conmuta la carga y la tierra.

Solución: PWM

Conmutar la carga muy rápido: la potencia promedio queda determinada por el porcentaje de tiempo en alto (ciclo de trabajo), no por el nivel de voltaje.

PARTE 1 · INTRODUCCIÓN

¿Qué es el PWM?

Definición y función

Periférico que genera una onda cuadrada de frecuencia y ciclo de trabajo programables, de forma autónoma, a partir de un contador y comparadores.

Problema que resuelve

Entregar potencia promedio variable a una carga (motor, LED, calefactor) con un pin digital y casi sin pérdidas.

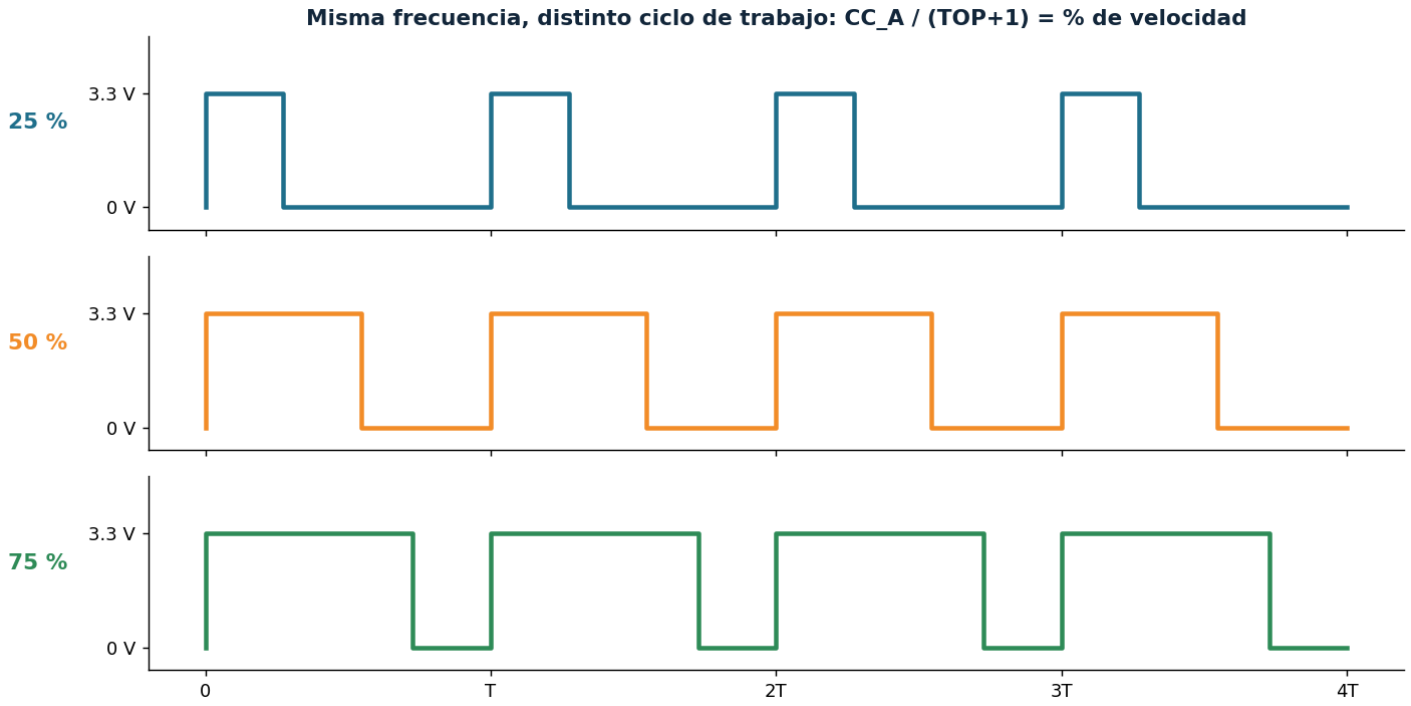
Tipo de información

Señal digital de dos niveles. El dato está en el tiempo: periodo T (frecuencia) y tiempo en alto (ciclo de trabajo D = t_alto / T).

Relación con CPU, buses y GPIO

La CPU escribe registros por el bus APB. Después el slice cuenta con clk_sys y la salida llega al pad vía IO_BANK0 (FUNCSEL = 4).

Ciclo de trabajo = velocidad



V_prom = D × Vs

Ejemplo con Vs = 9 V (ideal):

- 25 % → 2.25 V
- 50 % → 4.5 V
- 75 % → 6.75 V

```
f_PWM = clk_sys / (DIV × (TOP + 1))
```

Antes del microcontrolador

Método	Época	Cómo funcionaba	Limitación
Reóstato en serie	Inicios del s. XX	Resistencia variable en serie con el motor	Disipa potencia como calor, baja eficiencia
Ward-Leonard	1891	Generador variable alimenta el motor	Voluminoso, caro, mantenimiento
Troceador con tiristores	Desde ~1957	Conmutación de potencia con SCR	Circuito de apagado complejo
PWM analógico (comparador + triángulo, 555)	1970s	Se compara un nivel con una rampa	Deriva térmica, componentes discretos

Necesidad: control eficiente, preciso y repetible de potencia, integrado y programable.

PARTE 2 · ANTECEDENTES

Línea de tiempo

1891

Ward-Leonard

control de velocidad generador-motor

~1957

Tiristor (SCR)

troceadores de potencia

~1971

Timer 555

PWM analógico barato

~1978

MOSFET de potencia

conmutación rápida y eficiente

1980s

Drivers de puente H

L293 / L298 integrados

1990s

PWM en el MCU

timers/CCP como periférico

2021

RP2040

8 slices, divisor 8.4

2024

RP2350

12 slices, 24 canales

Evolución: qué mejoró

1	**Velocidad**	De cientos de Hz a decenas de kHz y más; la frecuencia se escoge por software (aquí 1–20 kHz).
2	**Integración**	De CI discretos (555, TL494) a un bloque dentro del microcontrolador.
3	**Precisión**	Reloj de cristal + contador de 16 bits: sin deriva térmica; 1000 pasos con TOP = 999.
4	**Consumo**	El transistor conmuta entre corte y saturación: pocas pérdidas frente al reóstato.
5	**Flexibilidad**	Frecuencia, ciclo, polaridad y modo cambian por registros, sin tocar el hardware.

Situación actual en sistemas embebidos

8 → 12

slices PWM (RP2040 → RP2350)

16 → 24

canales de salida

125 → 150

MHz de clk_sys por defecto

- Cada slice: contador de 16 bits, divisor fraccionario 8.4 y dos comparadores (canales A y B).
- Los slices son independientes: cada uno con su propia frecuencia.
- Una IRQ por wrap y solicitud de DMA para actualizar el ciclo sin CPU.
- También existe PIO para generar PWM a medida; no es el periférico de esta práctica.

Aplicaciones actuales

Motores y robótica

Ruedas de robots, ventiladores, ESC de drones.

Iluminación LED

Atenuación de LEDs y retroiluminación de pantallas.

Fuentes conmutadas

Convertidores DC-DC y cargadores: regulan con PWM.

Audio clase D

Amplificadores eficientes y DAC sencillos con filtro RC.

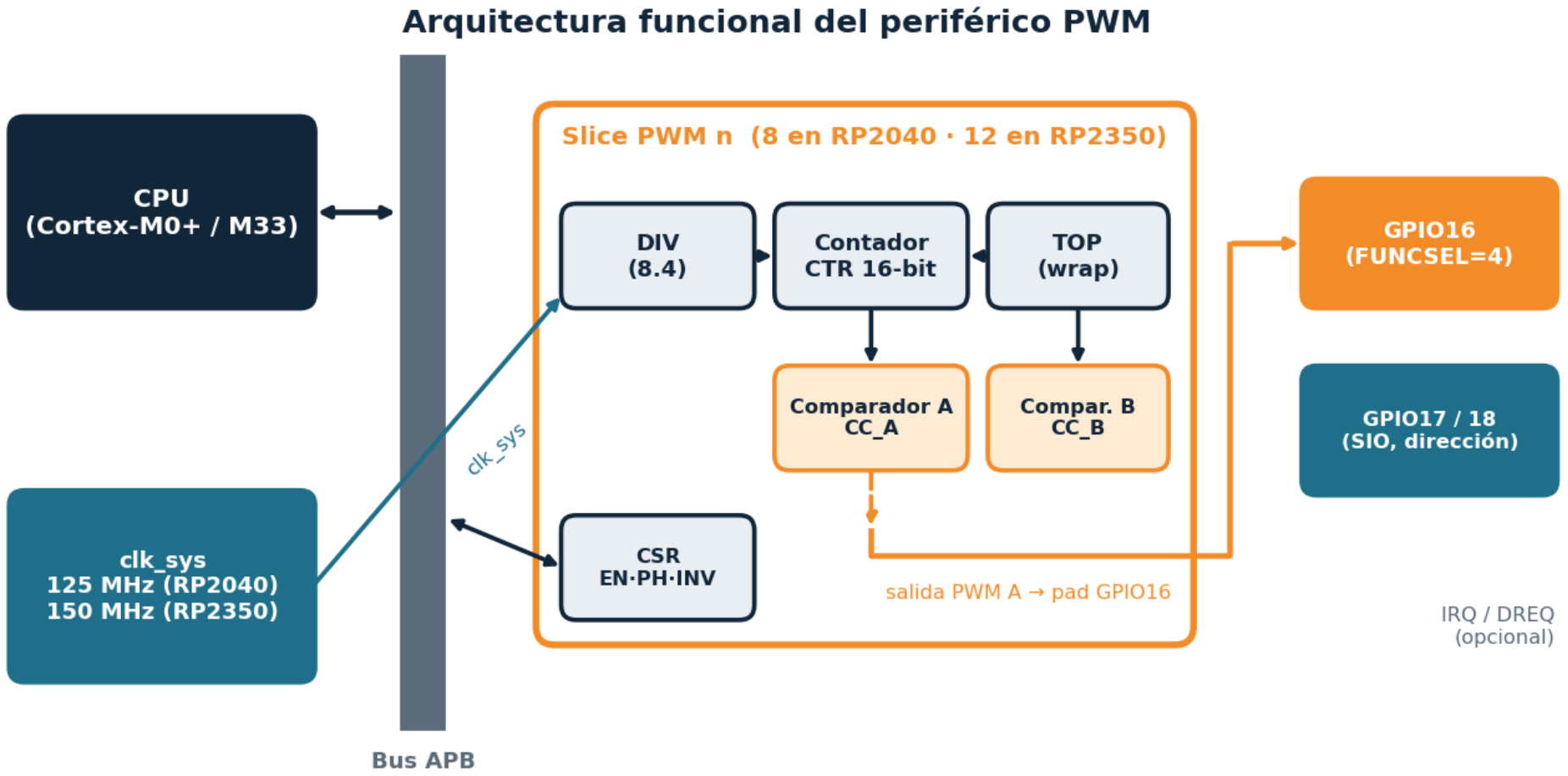
Servos y actuadores

Servomotores RC: el ancho del pulso define la posición.

Vehículos eléctricos

Inversores que modulan la corriente de motores trifásicos.

Arquitectura del periférico PWM



Bloques internos, E/S y reloj

Bloque / señal	Función
clk_sys (entrada)	Reloj del contador: 125 MHz (RP2040) / 150 MHz (RP2350)
DIV 8.4	Divide el reloj con parte fraccionaria (1/16)
CTR 16 bits	Cuenta 0 → TOP y reinicia
Comparadores A y B	Salida en alto mientras CTR < CC
DIVMODE (CSR[5:4])	0 libre · 1 nivel de B · 2 flanco ↑ de B · 3 flanco ↓ de B
IRQ / DREQ (salida)	Aviso de wrap; DMA para actualizar CC. No usado
GPIO (salida)	PWM_A / PWM_B al pad; FUNCSEL = 4

GPIO asociados: slice y canal

Slice	Canal A	Canal B
0	GP0 / GP16	GP1 / GP17
1	GP2 / GP18	GP3 / GP19
2	GP4 / GP20	GP5 / GP21
3	GP6 / GP22	GP7 / GP23
4	GP8 / GP24	GP9 / GP25
5	GP10 / GP26	GP11 / GP27
6	GP12 / GP28	GP13 / GP29
7	GP14	GP15

Nuestro pin

GP16 = slice 0, canal A

```
slice = (gpio >> 1) & 7
canal = gpio & 1
```

Atención

GP0 y GP16 comparten PWM0 A: usar solo uno. GP17 y GP18 funcionan como SIO (dirección), no como PWM.

Mapa de registros del slice 0

Registro	Offset	RP2040	RP2350	Acc.	Para qué sirve
CSR	`0x00`	`0x40050000`	`0x40058000`	R/W	Habilita y configura el slice
DIV	`0x04`	`0x40050004`	`0x40058004`	R/W	Divisor de reloj 8.4
CTR	`0x08`	`0x40050008`	`0x40058008`	R/W	Contador actual
CC	`0x0C`	`0x4005000C`	`0x4005800C`	R/W	Nivel de comparación A y B
TOP	`0x10`	`0x40050010`	`0x40058010`	R/W	Valor de wrap (periodo)
EN (global)	`0xA0` / `0xF0`	`0x400500A0`	`0x400580F0`	R/W	Un bit por slice

Slice n: PWM_BASE + n × 0x14 · Registros de 32 bits, alineados a 4 bytes

Bits y campos utilizados

CSR

bit 0 EN → 1 = corre
bit 1 PH_CORRECT → 0
bit 2 A_INV → 0
bit 3 B_INV → 0
bits 5:4 DIVMODE → 0

Usamos CSR = 0x1

DIV (8.4)

bits 11:4 INT
bits 3:0 FRAC (1/16)

reg = div × 16

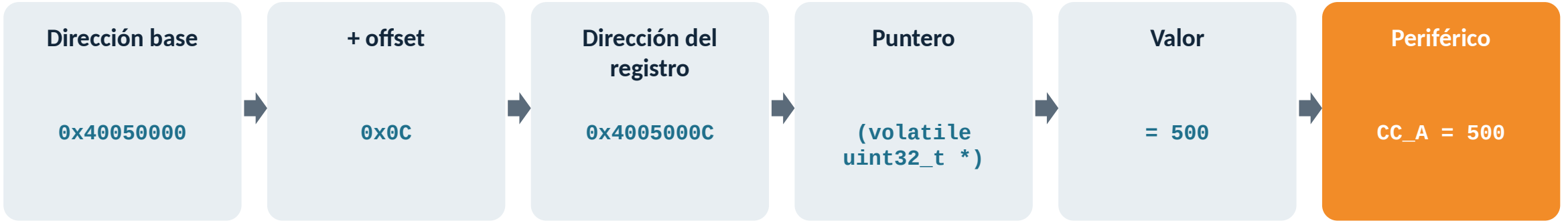
5 kHz:
RP2040 25.0 → 0x190
RP2350 30.0 → 0x1E0

CC y TOP

CC bits 15:0 → A
CC bits 31:16 → B
TOP bits 15:0

TOP = 999
CC_A = 500 → 50 %
CC_A = 1000 → 100 %
CC_A = 0 → 0 %

De la dirección al periférico




```
#define REG32(a)   (*(volatile uint32_t *)(uintptr_t)(a))
#define PWM_CC(s)  (PWM_BASE + (s)*0x14u + 0x0Cu)

REG32(PWM_CC(0)) = 500;  // *(0x4005000C) = 500
```

El mismo código funciona en RP2350 cambiando solo PWM_BASE (0x40058000).

Conceptos clave del acceso directo

volatile

El hardware puede cambiar el valor solo y escribir produce efectos. Evita que el compilador elimine, junte o reordene accesos.

Tamaño del registro

32 bits → uint32_t. Un acceso de otro ancho puede ignorarse o generar un fallo.

Máscaras y campos

v = (v & ~0x1F) | 4; → cambia solo FUNCSEL (bits 4:0) y respeta el resto de bits.

Alias atómicos

reg + 0x2000 → SET · reg + 0x3000 → CLR · reg + 0x1000 → XOR. Sin leer-modificar-escribir.

Inicialización paso a paso (registros)

1 **Salir de reset**

```
RESETS.RESET &= ~(IO_BANK0 | PADS | PWM)
esperar RESET_DONE
```

2 **Pines IN1/IN2**

```
FUNCSEL = 5 (SIO)
GPIO_OE_SET, GPIO_OUT_CLR
```

3 **Configurar slice**

```
CSR = 0 · TOP = 999
DIV = f(clk_sys) · CC = 0
```

4 **Conectar GP16**

```
RP2350: PADS.ISO = 0
GPIO16_CTRL.FUNCSEL = 4
```

5 **Habilitar**

```
CSR |= EN (alias SET)
ahora CC_A fija la velocidad
```

La misma función con el SDK

```
gpio_set_function(16, GPIO_FUNC_PWM);
uint slice = pwm_gpio_to_slice_num(16);  // 0

pwm_config c = pwm_get_default_config();
pwm_config_set_wrap(&c, 999);        // TOP
pwm_config_set_clkdiv(&c, 25.0f);    // 30.0f RP2350
pwm_init(slice, &c, true);           // CSR.EN = 1

pwm_set_gpio_level(16, 500);         // CC_A = 50 %
```

Qué abstrae

Direcciones base, offsets, resets, mapeo GPIO→slice/canal y el formato 8.4 de DIV.

Mismo código en ambos chips

El SDK elige el hardware según PICO_BOARD: pico (RP2040) o pico2_w (RP2350).

SDK y registros reales

Función del SDK	Registro / campo que escribe
`gpio_set_function(16, GPIO_FUNC_PWM)`	GPIO16_CTRL.FUNCSEL = 4 (y PADS.ISO = 0 en RP2350)
`pwm_gpio_to_slice_num(16)`	(16 >> 1) & 7 = slice 0 — solo cálculo
`pwm_config_set_wrap + pwm_init`	PWM_TOP y PWM_CSR del slice
`pwm_set_clkdiv(slice, 25.0f)`	PWM_DIV = 25 × 16 = 0x190
`pwm_set_gpio_level(16, 500)`	PWM_CC bits 15:0 (canal A)
`pwm_set_enabled(slice, true)`	PWM_CSR.EN = 1

Ventaja: portabilidad y menos errores. Limitación: oculta qué pasa en hardware y usa float para DIV.

PARTE 4 · OPCIONES

Opciones de configuración del PWM

Opción	Dónde	Para qué sirve	En el proyecto
Frecuencia	DIV (y TOP)	Sube/baja el tono y el rizado del motor	5 kHz; comando freq
Resolución	TOP	Número de pasos del ciclo de trabajo	999 → 0.1 %
Ciclo de trabajo	CC_A / CC_B	Define la velocidad promedio	comando motor
Phase-correct	CSR.PH_CORRECT	Cuenta ↑ ↓: pulso centrado, f/2	No (0)
Inversión	CSR.A_INV / B_INV	Cambia la polaridad de la salida	No (0)
Modo del divisor	CSR.DIVMODE	Usa B como entrada: mide ancho o frecuencia	Libre (0)
Corriente del pad	PADS.DRIVE	2/4/8/12 mA por pin	Por defecto
IRQ / DMA	INTE · DREQ	Aviso al terminar el periodo; DMA	No usado

PARTE 5 · COMPARACIÓN

RP2040 vs RP2350: panorama

Característica	RP2040 (Pico)	RP2350 (Pico 2 W)
Núcleos	2 × Cortex-M0+	2 × Cortex-M33 ó 2 × Hazard3 (RISC-V)
clk_sys por defecto (SDK)	125 MHz	150 MHz
SRAM	264 KB	520 KB
GPIO	30	30 en RP2350A (48 en RP2350B)
PWM	8 slices · 16 canales	12 slices · 24 canales
Seguridad	—	TrustZone-M, arranque seguro, OTP
SDK / placa	Pico SDK 1.x/2.x · PICO_BOARD=pico	Pico SDK ≥ 2.0 · PICO_BOARD=pico2_w

PARTE 5 · COMPARACIÓN

Qué cambia en registros y direcciones

Elemento	RP2040	RP2350
PWM_BASE	`0x40050000`	`0x40058000`
PWM_EN (global)	`+0xA0`	`+0xF0`
RESETS / IO_BANK0 / PADS_BANK0	`0x4000C000 / 0x40014000 / 0x4001C000`	`0x40020000 / 0x40028000 / 0x40038000`
Bits de reset: IO / PADS / PWM	`5 / 8 / 14`	`6 / 9 / 16`
PADS GPIOn bit 8 (ISO)	`No existe`	`Sale en 1 → limpiar`
SIO OUT_SET / OUT_CLR / OE_SET	`0x14 / 0x18 / 0x24`	`0x18 / 0x20 / 0x38`
DIV para 5 kHz (TOP = 999)	`25.0 → 0x190`	`30.0 → 0x1E0`

Cómo lo resolvimos en el código

```
#if PICO_RP2350
 #define PWM_BASE        0x40058000u
 #define RST_BIT_PWM     (1u << 16)
 #define PAD_HAS_ISO     1
 #define SIO_OUT_SET     0x018u
 #define PWM_SYSCLK_HZ   150000000u
#elif PICO_RP2040
 #define PWM_BASE        0x40050000u
 #define RST_BIT_PWM     (1u << 14)
 #define PAD_HAS_ISO     0
 #define SIO_OUT_SET     0x014u
 #define PWM_SYSCLK_HZ   125000000u
#endif
```

Registros

chip_map.h con #if por chip. El resto de pwm_regs.c es único.

SDK

pwm_sdk.c sin #if. Se compila con -DPICO_BOARD=pico o pico2_w.

Comando motor: la cadena completa



Comando	Efecto físico
`motor 50 derecha`	IN1=1, IN2=0, CC_A = 500 → gira a la derecha al 50 %
`motor 80 izquierda`	IN1=0, IN2=1, CC_A = 800 (pausa de 150 ms si venía en sentido contrario)
`motor parar` · `motor freno`	ENA=0 (rueda libre) · IN1=IN2=1 con ENA=100 % (frenado)

argc, argv y validación

argv	Contenido
`argv[0]`	`"motor"`
`argv[1]`	`"50"  (texto → strtol → 0..100)`
`argv[2]`	`"derecha" | "izquierda"`

Validaciones

- Número de argumentos (argc)
- Formato numérico y rango 0–100
- Sentido válido (derecha / izquierda)
- Frecuencia 1000–20000 Hz
- Error → imprime uso, no toca el hardware

Subcomandos

parar · freno · freq <Hz> · modo reg|sdk · rampa <%> <ms> · info

PARTE 6 · REPL

Código del comando

```
int cmd_motor(int argc, char **argv) {
  uint32_t v; motor_dir_t d;
  if (argc < 2) return usage();

  if (parse_u32(argv[1], 0, 100, &v) == 0) {
    if (argc != 3 || parse_dir(argv[2], &d))
      return usage();
    apply(d, v);   // IN1/IN2 + CC_A
    return 0;
  }
  /* parar | freno | freq | modo | rampa | info */
  ...
}
```

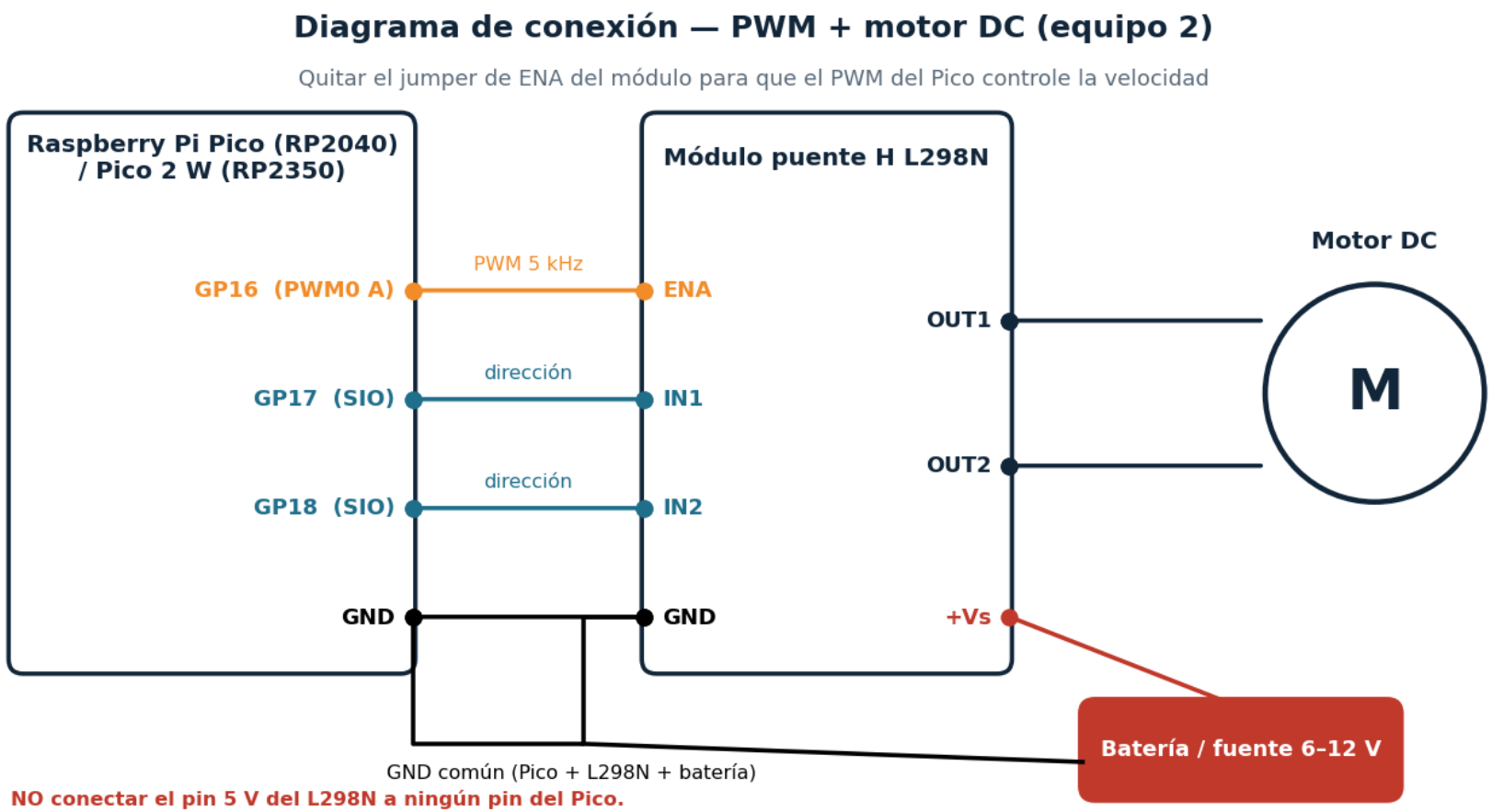
Interfaz común

pwm_backend_t: init, set_freq, set_level, set_dir, dump.

Dos implementaciones

pwm_regs.c (punteros) y pwm_sdk.c. motor modo reg|sdk las intercambia.

Diagrama de conexión



Componentes y precauciones

Componentes

- Raspberry Pi Pico (RP2040)
- Raspberry Pi Pico 2 W (RP2350)
- Módulo puente H L298N
- Motor DC 6–12 V
- Fuente externa 6–12 V
- Capacitor 100 nF, protoboard, jumpers

Precauciones

- Tierra común Pico + L298N + fuente
- Quitar el jumper de ENA
- No conectar el 5 V del L298N al Pico
- Fuente del motor apagada al cablear
- Capacitor de 100 nF en el motor
- Pausa de 150 ms al invertir el sentido

PARTE 7 · DEMOSTRACIÓN

Guion de la demostración

1	`chip`	Confirma RP2040 o RP2350 y la base del PWM
2	`motor info`	Muestra CSR, DIV, TOP y CC del slice 0
3	`motor 30 derecha` → `motor 70 derecha`	La velocidad aumenta; CC_A pasa de 300 a 700
4	`motor 50 izquierda`	Pausa de 150 ms y el motor invierte el giro
5	`motor modo sdk` → `motor info`	Mismos valores de registros con el SDK
6	`motor freq 1000` · `motor freq 20000`	Cambia el tono; DIV se reescribe
7	`motor freno` · `motor parar`	Frenado y rueda libre

PARTE 7 · DEMOSTRACIÓN

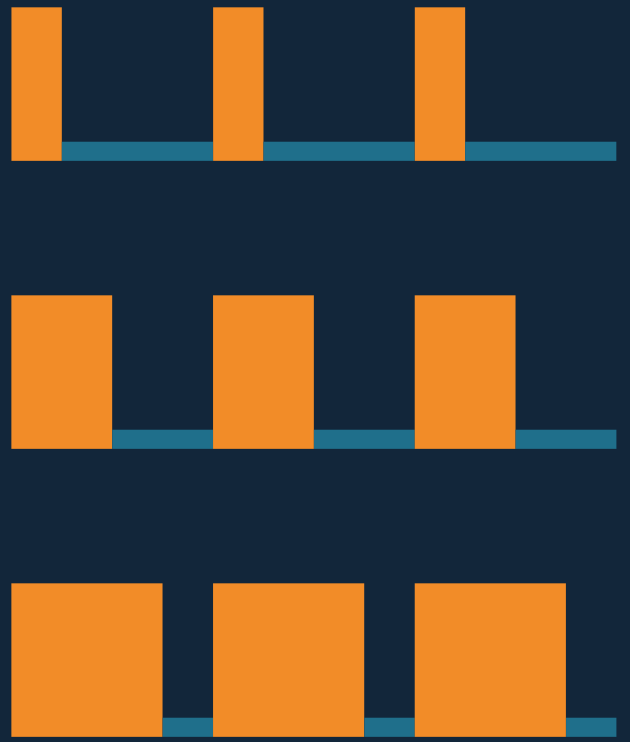
Qué debe mostrar la demo

Medida tras: motor 50 derecha @ 5 kHz	RP2040	RP2350
PWM0_CSR	`0x00000001`	`0x00000001`
PWM0_DIV	`0x00000190 (25.0)`	`0x000001E0 (30.0)`
PWM0_TOP	`999`	`999`
PWM0_CC	`0x000001F4 (A = 500)`	`0x000001F4 (A = 500)`
Señal en GP16	`5 kHz · 50 %`	`5 kHz · 50 %`
IN1 / IN2	`1 / 0`	`1 / 0`

Resultado esperado según el diseño; contrastar con las capturas en /evidencias.

Conclusiones

- El PWM convierte un pin digital en control de potencia eficiente.
- Con punteros se ve el hardware real: base + offset → registro → valor.
- El SDK ofrece portabilidad, pero escribe los mismos registros.
- RP2040 → RP2350: cambian bases, resets, pads (ISO), SIO y clk_sys; la lógica del PWM es la misma.
- El comando REPL integra todo: comando → C → PWM → L298N → motor.



Referencias técnicas

- Raspberry Pi Ltd. RP2040 Datasheet (PWM, RESETS, IO_BANK0, SIO). datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf
- Raspberry Pi Ltd. RP2350 Datasheet (PWM, RESETS, IO_BANK0, SIO). datasheets.raspberrypi.com/rp2350/rp2350-datasheet.pdf
- Raspberry Pi Ltd. Raspberry Pi Pico-series C/C++ SDK. datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf
- Raspberry Pi Pico SDK API: hardware_pwm. raspberrypi.com/documentation/pico-sdk
- STMicroelectronics. L298 Dual Full-Bridge Driver, Datasheet.
- Wokwi Docs. Raspberry Pi Pico (wokwi-pi-pico). docs.wokwi.com/parts/wokwi-pi-pico
- Repositorio del proyecto: código, mapa de registros, diagramas y evidencias.

Preguntas

Ana Cisneros · Leslie Sanchez

PWM + motor DC · RP2040 y RP2350